You said:

TermMap A Clinic...
Document

go through this problem statement in detail, and also analyse web sources and practical workaround and assumptions based on a deep research then help me answer a few questions before we start working on its technical implementation and architecture
Web search dont provide me a very verbose analysis and long descriptions and talk in points or diagrams and not huge paragraphs to present the important points and help me understand this ps better

ChatGPT said:

TermMap — Problem Statement Analysis

The objective is to build a clinical term mapping engine for HiLabs that converts extracted medical terms into **both ICD-10-CM codes and SNOMED CT concept IDs**, targeting more than 95% exact-match accuracy on each system.

I'll use the uploaded problem statement as the primary specification, distinguish its explicit requirements from practical assumptions, and use external sources to validate potential approaches. We can then work through your questions before committing to an architecture or implementation.

1. What exactly are we building?

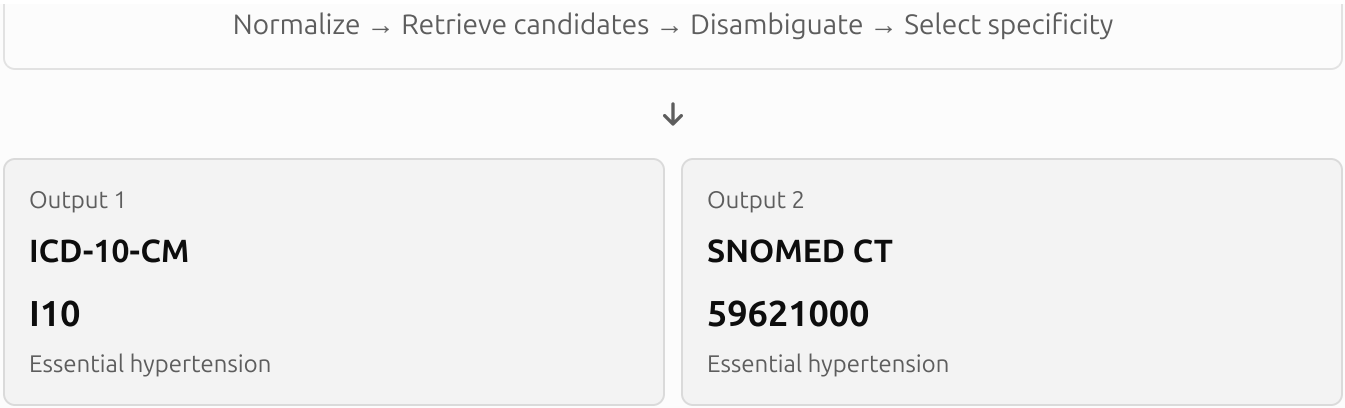
Input: Extracted clinical entity

Term + chunk ID

Example: `HTN, well controlled`

↓

TermMap engine



Illustrative mapping based on the problem statement's example. Both outputs must be predicted independently; the engine must not simply derive one from the other.

Important scope boundaries

- Input terms are already extracted. We do not need to build medical entity recognition.
- We receive the term and its chunk context, not the full patient chart.
- Each input needs exactly one ICD-10-CM code and one pre-coordinated SNOMED CT concept ID.
- Predictions must come from the supplied dictionaries.
- The system must process the evaluation file automatically, without manually correcting its individual rows. TermMap A Clinical Term Mapping Engine for SNOMED 3fc8675f56b3808d9d69c6f6fdc631c6.md

2. The real difficulty: why 95% is hard

The problem statement identifies seven failure modes. The most important ones are:

Challenge	Example	What the engine must do
Abbreviations	`MS`	Distinguish multiple sclerosis from mitral stenosis using context
Similar names	Hypertension vs pulmonary hypertension	Avoid choosing a lexically similar but clinically wrong concept
Specificity	`Diabetes`	Avoid inventing complications not stated in the text
Noisy text	`chronickidney disease stag 3`	Normalize spelling and OCR errors
Rare conditions	Unseen or uncommon diagnoses	Generalize beyond memorized training examples

Different code systems	SNOMED vs ICD-10-CM	Select each target independently
Version drift	New or revised codes	Respect the supplied code dictionaries and their versions

The key insight: **candidate retrieval is not the same as correct code selection.** A system can retrieve the right concept among its top candidates and still choose the wrong code because of ambiguity or specificity.

3. What external research tells us

These are the most relevant findings from official documentation and published research.

Official terminology resources

The September 2026 SNOMED CT US Edition includes the SNOMED-to-ICD-10-CM reference set, with over 142,000 source concepts mapped to targets. This is valuable candidate-generation and validation data, but it does not eliminate the need to predict both outputs correctly. National Libr… +1

Resource: NLM — SNOMED CT US Edition ↗

Biomedical retrieval models

MedCPT provides a biomedical retriever and a cross-encoder reranker. Its published results support the retrieve-then-rerank design pattern, although its performance on biomedical literature retrieval does not establish 95% accuracy on this particular coding dataset. PubMed +1

Paper: MedCPT — Bioinformatics ↗

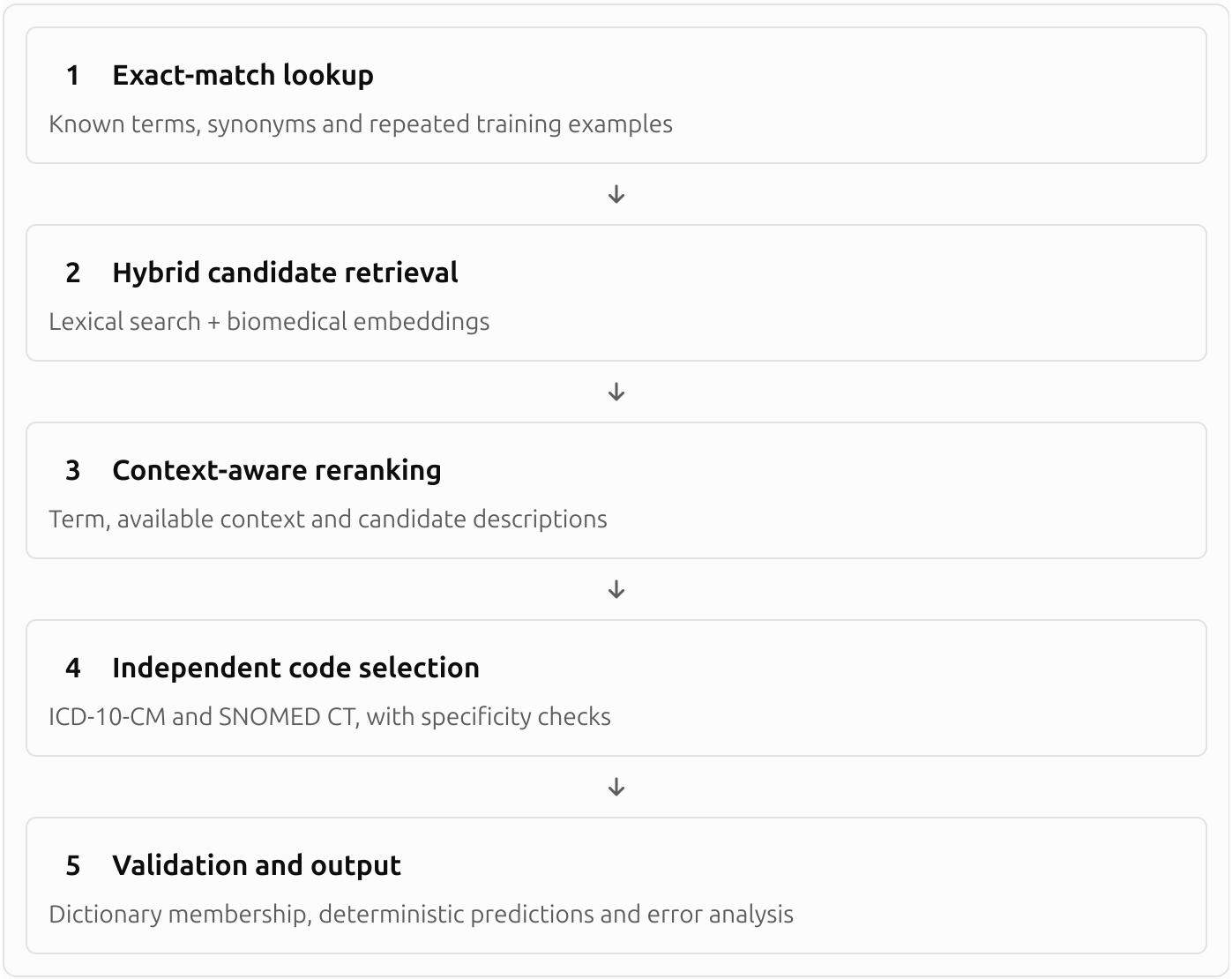
ICD-10-CM rules and versioning

The official coding guidelines and tabular instructions matter when deciding specificity and interpreting exclusions. The CDC's FY2027 release became effective October 1, 2026, so version alignment is especially important for this challenge. ftp.cdc.gov +1

Resource: CDC — ICD-10-CM files ↗

4. Practical approach worth evaluating

This is a proposed direction, not a finalized architecture.



Practical considerations:

- **Do not start with an LLM call for every row.** Exact lookup and retrieval are cheaper and easier to reproduce.
- **Do not trust embeddings alone.** Negation, laterality, disease subtypes and closely related concepts need explicit evaluation.
- **Use the labelled training data to learn challenge-specific conventions.** Repeated mappings and synonyms may provide a strong baseline.
- **Treat the crosswalk as a prior, not ground truth.** The official mapping can have conditional rules and one-to-many relationships. National Libr… +1
- **Benchmark before adding complexity.** Measure each system separately and investigate the errors contributing to the lower accuracy.

5. Important questions and assumptions to settle

There is one particularly important ambiguity in the problem statement.

> **Potential issue: where is the chunk context?**
>
> The narrative says the engine must use surrounding chunk context. However, the listed training columns are only `chunk_id`, `clinical_entity`, `icd10cm_code`, and `snomed_code`. A chunk identifier is not the actual clinical text.
>
> **Practical implication:** if no separate file supplies the chunk text, the engine may have to operate mostly on the entity itself, perhaps exploiting other entities sharing the same chunk ID. We should inspect the actual CSV before assuming contextual disambiguation is possible.

Other questions worth resolving before implementation:

1. What are the training dataset size, label distribution and frequency of repeated entities?
2. Does the evaluation file include chunk context anywhere beyond the listed columns?
3. Are training and evaluation splits separated by rows, chunks or patients? This affects leakage and validation.
4. Do the supplied Excel dictionaries contain synonyms, active/inactive flags and descriptions, or only code-description pairs?
5. Is the greater-than-95% target an absolute requirement, or is the final ranking relative to other teams? The document states the target but does not establish a guaranteed achievable result.
6. Can external pretrained models and terminology resources be downloaded and run locally within the available time and compute budget?

6. Suggested order of work

Given the stated deadline of **October 18, 2026**, I recommend this sequence:

- **First:** inspect the CSV and both Excel dictionaries; establish the exact input/output schema.
- **Second:** build a simple baseline and measure exact-match accuracy separately for both systems.
- **Third:** examine errors by category before deciding whether to prioritize normalization, retrieval, context or supervised reranking.
- **Fourth:** evaluate each addition on a held-out validation set, keeping the evaluation data untouched.

- **Fifth:** finalize reproducible inference, error analysis and the required submission artifacts.

The deliverables include a working engine, predictions, a private GitHub repository, a 2–3 page write-up, a documentation PDF and a short demo video.

TermMap A Clinical Term Mapping Engine for SNOMED 3fc8675f56b3808d9d69c6f6fdc631c6.md

Next step: Ask your questions about the problem statement, constraints, assumptions or possible workarounds. We can resolve those individually before choosing the technical architecture.

You said:

SNOMEDCT_US_c... Spreadsheet

icd10cm_codes.xlsx Spreadsheet

hackathon_train.csv Spreadsheet

i have also attach the datasets and files given to us analyze the files too and establish relations between the columns for each file and help me clarify what will be given and around what do we need to build our architecture, dont scan the full doc the first few rows would be fine for now

Worked for 2m 39s

Analyzed

Analysis errored

Analyzed

Analyzed

Analyzed

ChatGPT said:

1. What the actual files contain

I inspected the first few rows of the uploaded CSV and the SNOMED workbook, and checked the ICD workbook's initial entries. I have not scanned the complete datasets.

The most important finding is that **the actual training CSV has a different schema from the one described in the problem statement.** This affects how we should design the input pipeline.

File 1 — `hackathon_train.csv`

The CSV has these six columns:

Column	Example / meaning
`entity`	Extracted term, e.g. `+2 bilateral lower extremity edema`
`chunk_text`	Surrounding clinical text or table content
`coding_system`	`ICD10CM` or `SNOMEDCT_US`
`code`	Gold-standard code for that row
`display`	Human-readable description of the gold code
`subconcept`	Category such as `DiagnosisCode`, `LabService`, or `ProcedureCode`

Example rows from the file:

Example A — same clinical entity, two separate labels

Entity: `+2 bilateral lower extremity edema`

Coding system	Code	Display
ICD10CM	R60.0	Localized edema
SNOMEDCT_US	816181009	Bilateral lower limb edema

The two rows demonstrate that one entity can have one gold label per coding system, with different levels of specificity.

The `chunk_text` column contains context such as clinical statements, tabular data, and sometimes OCR/HTML-like fragments. This is important because it means the training data does provide context directly, rather than just a chunk identifier.

A significant inconsistency: training scope

In the first 1,000 CSV rows, I found these `subconcept` values:

`subconcept`	Rows in first 1,000
`DiagnosisCode`	505
`ProcedureCode`	259
`EncounterType`	93

`LabService`	84
`Device`	59

These counts are only from the inspected sample, not the complete dataset.

The problem statement says that only diagnoses and conditions are in scope, but the CSV contains other categories too.

Implication: we should not blindly train on every row as though it were a diagnosis. We need to establish whether the intended approach is to filter to `DiagnosisCode` or whether the organizers expect a different interpretation of the dataset. We should also inspect the evaluation input schema before deciding.

File 2 — `SNOMEDCT_US_codes_and_descriptions.xlsx`

The workbook has three sheets:

- `SNOMED codes`
- `Allergy reaction labels`
- `Notes`

The main `SNOMED codes` sheet has three columns:

Column	Meaning
`snomed_code`	SNOMED CT concept identifier
`description`	A name or synonym for the concept
`description_order`	Ordering of descriptions associated with a concept

Example rows:

`snomed_code`	`description`	`description_order`
58488005	1,4-alpha-Glucan branching enzyme	1
58488005	Branching enzyme	2
58488005	Amylo-(1,4,6)-transglycosylase	3

Notice that **multiple descriptions can share the same SNOMED code**. These are useful for synonym-based retrieval.

Practical implication:

- Do not treat every description as a different concept.
- Build a mapping from each concept ID to its available descriptions.
- Index the descriptions for retrieval, but return the corresponding concept ID.
- Do not assume `description_order = 1` is necessarily the best description for every clinical term.

The other two sheets may provide additional information, but I have not yet inspected their full contents.

File 3 — `icd10cm_codes.xlsx`

The workbook's initial entries have two columns:

Column	Meaning
`Code`	ICD-10-CM code or category identifier
`Description`	Text describing the code or category

Examples from the first rows:

Code	Description
A00	Cholera
A00-B99	Diseases generally recognized as communicable or transmissible
A00.0	Classical cholera
A00.0	Cholera due to *Vibrio cholerae* 01, biovar cholerae

Notice that even this small sample contains a broad category, a subcategory and multiple descriptions associated with the same code.

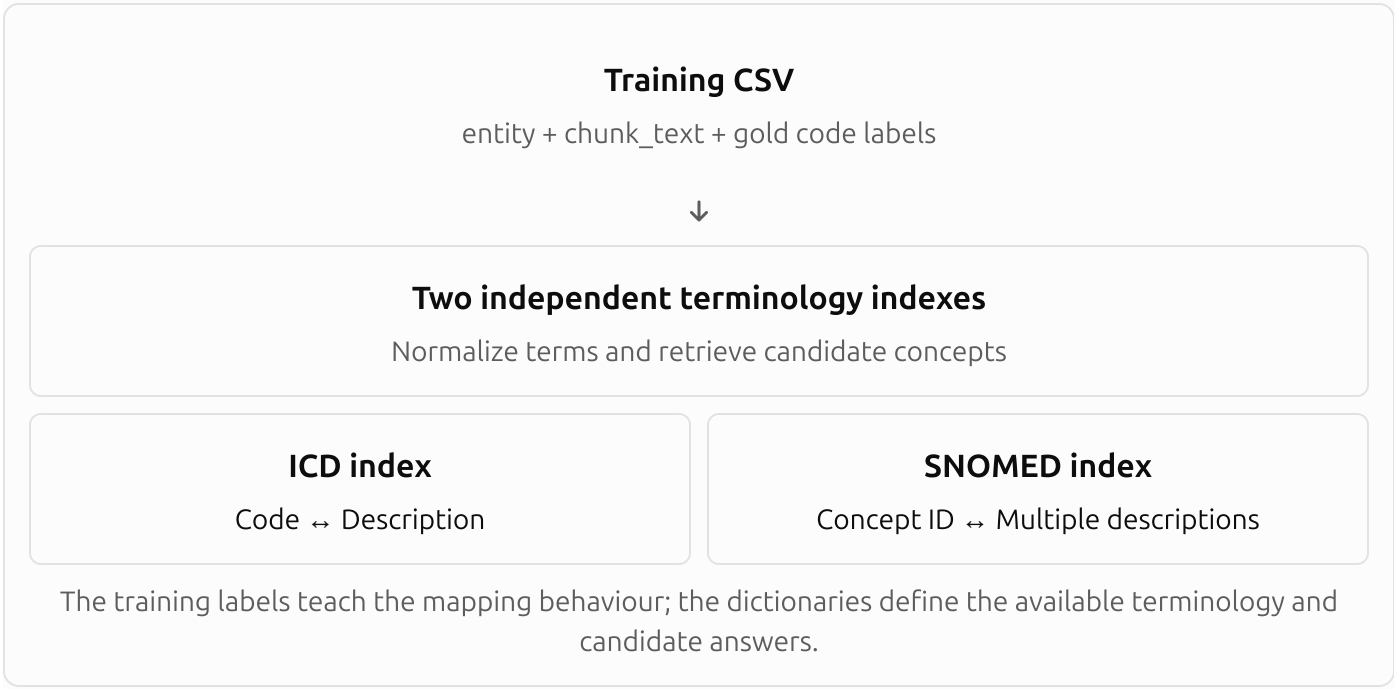
Practical implication:

- Keep codes as strings, not numeric values.

- Deduplicate descriptions only when appropriate; preserve multiple descriptions as retrieval aliases.
- Check whether a candidate is a valid target under the challenge's supplied dictionary rules.
- Do not assume every dictionary entry is a billable leaf code merely because it appears in the workbook.

I have not established the full code hierarchy or validated every entry.

2. How the three files relate



One subtlety: the CSV's `display` is the description of the **gold label**, whereas the dictionary descriptions are the searchable terminology. They may not use identical wording.

The CSV also has a separate row for each coding system, so we should not assume that the input will always arrive in paired rows. The inference pipeline must support the actual evaluation schema.

3. What will be given, and what must we build?

Based on the supplied problem statement and the files inspected so far:

Component	Supplied by the challenge	Our responsibility
Clinical entity	Yes, in the training data	Normalize noisy terms

Clinical context	Present as `chunk_text` in training CSV	Use it to disambiguate terms
ICD terminology	ICD code-description workbook	Build candidate retrieval and selection
SNOMED terminology	Concept-description workbook	Build synonym-aware candidate retrieval and selection
Gold mappings	`code` and `display` in training data	Learn and evaluate mapping behaviour
Final predictions	Not yet verified for evaluation format	Produce required codes deterministically
Cross-system mapping	Not established by these three files alone	Optionally use an approved crosswalk as supporting evidence

4. Three decisions to resolve before architecture

1. **Scope:** Should we filter training rows to `subconcept == "DiagnosisCode"`? The problem statement suggests yes, but the supplied CSV contains other categories.

2. **Input format:** Will evaluation rows contain `entity` and `chunk_text`, or only `chunk_id` and `clinical_entity` as described in the problem statement? This determines how much context our engine can use.

3. **Target schema:** Will each evaluation entity require two output codes in one row, or will it have separate rows per coding system? The training file uses separate rows per system, while the stated deliverable requests both codes.

My recommendation is to resolve these three questions first, then inspect the CSV's full schema and label distribution as needed. We do not need to scan every dictionary entry before deciding the architecture; the first few rows already establish the main relationships and several important data-quality considerations.

‹/›